

L3

PROIECTAREA BAZELOR DE DATE

1. Obiective

1. Proiectarea
2. Modelul entitate-relație. Obiectele bazelor de date relaționale
3. Construcția schemelor relație
 - 3.1. Relația „unul-la-unul” (1-1 sau one to one)
 - 3.2. Relația „unul-la-multe” (1-N sau one to many)
 - 3.3. Relația „multe-la-multe” (M-N sau many to many)
 - 3.4. Relația unară
4. Diagrama entitate-relație
5. Constrângeri de integritate
 - 5.1. Constrângerile de domeniu
 - 5.2. Constrângerile referitoare la n-upluri (înregistrările din tabelă)-Cheia primară
 - 5.3. Constrângeri între tabele
6. Dependente funcționale
7. Normalizare. Forme normale
 - 7.1. Forma normală de ordin 1 (FN1)
 - 7.2. Forma normală de ordin 2 (FN2)
 - 7.3. Forma normală de ordin 3 (FN3)
 - 7.4. Forma normală Boyce-Codd (FNBC)
8. Structuri de indecși în tabelele de date
 - 8.1. Indexul primar
 - 8.2. Indexul secundar
 - 8.3. Indexul de grup
 - 8.4. Indexul multinivel

2. Considerații teoretice

2.1. Ce este proiectarea?

Proiectarea unei baze de date constă din proiectarea logică și fizice a acesteia, pentru a corespunde cerințelor utilizatorilor pentru un anumit set de aplicații.

În general, vom considera că proiectarea corectă a unei baze de date trebuie să parcurgă următoarele etape:

- Analiza cererilor și strângerea de informații referitoare la aplicație;
- Proiectarea conceptuală a bazei de date;
- Alegerea unui sistem de gestiune al bazelor de date;
- Proiectarea logică a bazei de date;
- Proiectarea fizică a bazei de date;
- Implementarea bazei de date și a aplicației.

Înainte de a se proiecta efectiv o bază de date, este necesar să se cunoască ce rezultate se așteaptă potențialii utilizatori să obțină de la baza de date respectivă și documentarea asupra informațiilor ce sunt disponibile pentru aceasta. De asemenea, este necesară și definirea cât mai exactă a aplicației

(*De exemplu:* aplicație de gestiune a stocurilor, a cărților într-o bibliotecă, aplicație contabilă, aplicație folosită la salarizare, aplicație referitoare la gestiunea studenților, a profesorilor, a cursurilor, a notelor dintr-o instituție de învățământ etc.).

Având cerințele formulate precis și concis se poate trece la elaborarea schemei conceptuale utilizând un model de nivel înalt.

- a) **Schema conceptuală** reprezintă o descriere concisă a datelor utilizatorului, incluzând descrierea detaliată a tipurilor de date, a relațiilor și restricțiilor acestora. Deoarece până la acest moment nu se includ detalii de implementare, rezultatele pot fi comunicate utilizatorilor, chiar dacă sunt nespecializați în domeniu, și analizate de aceștia pentru eliminarea eventualelor conflicte care pot apărea.
- b) Următoarea etapă este cea de **construcție a bazei** de date ținând cont de detaliile actuale, cu ajutorul unui sistem de gestiune a bazelor de date (SGBD).

Această etapă înglobează etapele de:

- alegere a unui SGBD, de proiectarea logică a bazei de date;
- cea de proiectarea fizică a bazei de date.

Alegerea SGBD se face în funcție de complexitatea aplicației, de capacitatea de stocare a datelor, de posibilitatea refacerii datelor, de numărul de utilizatori care vor folosi aplicația, dar și de costurile de achiziție ale sistemului, de costurile de întreținere etc.

- c) Următoarea fază, cea de **proiectare logică** poate fi realizată în două sub-faze:

- transpunerea schemei conceptuale în modelul de date al sistemului SGBD ales, dar independent de sistemul de gestiune propriu-zis,
- sau rafinarea schemei conceptuale și a schemelor externe obținute anterior, astfel încât să se utilizeze mai multe din facilitățile oferite de sistemul SGBD ales (modul de generare a cheilor primare, definirea constrângerilor, etc.).

Aceste două sub-faze se pot realiza împreună, folosind unul din instrumentele de proiectare oferite de sistemul SGBD ales.

Rezultatul acestei faze de proiectare îl constituie, așadar, schema conceptuală și schemele externe ale bazei de date, dependente de sistemul SGBD ales și de modelul de date al acestuia.

- d) **Proiectarea fizică** a bazei de date reprezintă procesul de alegere a structurilor de memorare și de acces la fișierele bazei de date, pentru a obține performanțe cât mai bune pentru aplicația proiectată.

Ca parametrii generali de alegere a opțiunilor proiectului fizic al unei baze de date relaționale se pot enumera: timpul de răspuns, utilizarea spațiului de memorare, capacitatea tranzacțională.

Deciziile de proiectare fizică se pot lua numai după o analiză a aplicațiilor care se vor executa și în principal, a interogărilor și tranzacțiilor pe care acestea le vor lansa. În urma analizei se pot sintetiza informații care să dea imaginea de ansamblu a utilizării atributelor relațiilor bazei de date: care attribute sunt actualizate cel mai frecvent, care attribute sunt folosite cel mai frecvent în selecții ale interogărilor, etc. Aceste informații se folosesc pentru stabilirea indecșilor secundari ai relațiilor.

- e) Ultima etapă este cea de implementare efectivă a bazei de date și a aplicației. Aici se crează pe baza modelului definit obiectele bazei de date, se populează cu date baza de date, se verifică constrângerile, se crează interfețele cu utilizatorul și rapoartele neceare cu datele extrase din baza de date.

Exemplu:

Să se proiecteze și să se realizeze o aplicație simplificată pentru gestiunea informațiilor referitoare la notele obținute de studenții dintr-o universitate la anumite materii.

Baza de date se va numi **Universitate**.

Baza de date va fi rafinată la modelul entitate-relație, un model conceptual de nivel înalt. Acest model asigură perceperea de către utilizatori fără să prezinte detaliile de stocare a datelor păstrate în calculator.

În faza preliminară, după analiza cerințelor *se cunosc următoarele informații:*

- această universitate are în componență mai multe *facultăți*.

Fiecare *facultate* având asociat *un cod, o denumire, o adresă*.

- *studenții* au stocate în baza de date informațiile personale ale fiecăruia (cnp, nume, prenume, inițiala tatălui, data nașterii etc), dar și informații legate de starea actuală a lor (grupa în care se află, facultatea de care aparține etc.)

- în această bază de date stocăm și *materiile studiate* în facultățile din acea universitate. Se consideră materii diferite acele materii care au aceeași denumire, dar profesor diferit.

- vom stoca și *notele* obținute de fiecare student la materia la care a fost evaluat prin examen.

2.2. Modelul entitate-relație. Obiectele bazelor de date relaționale

Modelul entitate-relație este cel mai utilizat model conceptual de nivel înalt, care reprezintă schema conceptuală a bazei de date cu ajutorul entităților și a relațiilor dintre acestea. Acest model a fost introdus în anul 1976 de P.S.Chen. Elementele de bază folosite în cadrul acestui model sunt conceptele de entitate și cel de relație.

O *entitate* este un obiect al lumii reale, cu o existență independentă și poate reprezenta un obiect fizic, o activitate, un concept.

O entitate este un obiect cu existență fizică, de exemplu:
persoană particulară, automobil, companie, activitate, curs universitar.

Orice entitate are o serie de proprietăți numite *attribute*, ce descriu entitatea respectivă.

Cu toate că nu reprezintă același lucru, pentru denumirea de entitate se mai folosește și denumirea de tabel al bazei de date, iar pentru attribute câmpurile tabelului.

Scurte definiții a noțiunilor pe care le-am folosit și le vom folosi în continuare sunt :

- *Tabelă (entitate) este o colecție de informații logice relaționale tratată ca o unitate.*
- *Înregistrare (n-uplu). O tabelă este compusă din înregistrări sau rânduri. Fiecare înregistrare este tratată ca o simplă unitate. Fiecare înregistrare este legată de înregistrări ale altei tabele.*
- *Câmpuri (atribute). Înregistrările sunt constituite din câmpuri (coloane). Un câmp este o particulă atomică a bazei de date ce reprezintă cea mai mică cantitate de informație care poate fi manipulată. Toate înregistrările dintr-o tabelă au aceleași câmpuri.*

De **exemplu**, pentru baza de date **Universitate** pe care dorim să o creăm,

- o entitate reprezintă o
 - o **Facultate** cu atributele **Cod Facultate, Denumire, Adresa, Nume Decan.**
- O altă entitate ar fi:
 - o **Materii**, descrise de atributele **Cod Materie, Denumire, An, Nume Profesor.**

Atributelor li se asociază valori care au ca scop identificarea entității. Această atribuire de valori pentru fiecare atribut formează o **înregistrare** a tabelului respectiv.

Unele atribute pot fi divizate în mai multe părți cu semnificație independentă. Un astfel de atribut este un **atribut complex**.

Un **exemplu** este cel al atributului **Adresă** care poate fi divizat în mai multe atribute: **Oraș, Cod Postal, Stradă, Număr, Bloc** etc. sau al atributului **Nume Decan** care poate fi divizat în **Nume** și **Prenume**.

Atributele care nu sunt compuse se numesc **atribute atomice**.

Valoarea atributelor complexe se formează prin concatenarea valorilor atributelor atomice.

Multe atribute au *valoare unică* pentru o entitate particulară și sunt numite **atribute cu o singură valoare**.

De **exemplu** CNP-ul unei persoane.

Există **atribute ce pot lua mai multe valori** dintr-un set dat.

Exemplu gradele didactice ale profesorilor universitari, culorile etc. Aceste atribute sunt atribute cu mai multe valori.

Atributele derivate sunt atributele ce se pot determina din alte atribute.

Exemplu vârsta unei persoane se poate calcula din data curentă minus data nașterii persoanei respective.

În anumite situații, o entitate poate să nu aibă valori pentru toate atributele asociate ei, în acest caz folosindu-se o valoare specială numită **atributul null**.

Un **exemplu** ar fi lipsa din atributul Adresă a numelui blocului sau a scării, etc.

Exemplu:

Revenind la exemplu nostru, să ne definim toate entitățile din baza de date Universitate.

Entitățile ar putea fi următoarele:

- **Facultate** cu atributele: **CodFac**, Denumire, Adresa, Nume Decan
- **Studenti** cu atributele: **CodStud**, An, Grupa, Media, Bursa
- **StudPersonal** cu atributele: **CNP**, Nume, Init, Prenume, Data Nasterii, Loc Nașterii, Tata, Mama, Adresa
- **Materii** cu atributele: **Cod Materie**, Denumire, An, Nume Profesor
- **Note** cu atributele Nota, Data

2.3. Construcția schemelor relație

În proiectarea bazelor de date se definesc relații sau asocieri între mulțimile de entități componente, pentru a reprezenta anumite aspecte ale realității pe care o modelează baza de date.

O *relație* este o corespondență între entități din una sau mai multe mulțimi de entități. Gradul unei relații este dat de numărul de mulțimi de entități asociate. Relațiile pot fi:

- binare (între 2 mulțimi de entități) sau
- multiple (între mai mult de 2 entități).

Relațiile **binare** sunt împărțite în *trei categorii*, după numărul elementelor din fiecare din cele două mulțimi puse în corespondență de relația respectivă.

Se consideră 2 mulțimi de entități E1 și E2.

2.3.1. Relația „unul-la-unul” (1-1 sau one to one)

Relația „unul-la-unul” este cel mai simplu tip de relație. Ea este relația prin care unui element din mulțimea E1 îi corespunde un singur element din mulțimea E2 și reciproc (Figura 3.1.).

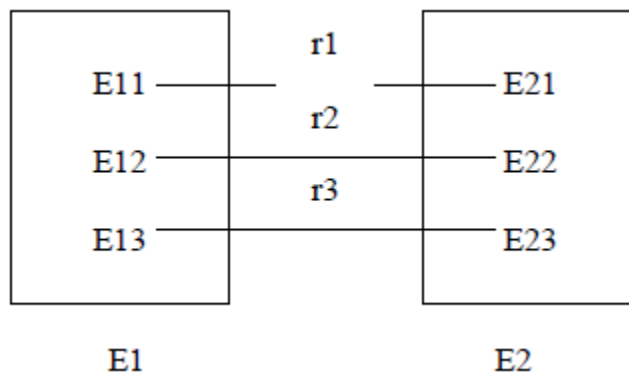


Figura 3.1. Relația „unul-la-unul”

Această relație „unul-la-unul” este foarte rar folosită în lumea reală. Cel mai des, ea este folosită pentru a reduce numărul de atribute dintr-o entitate, pentru a nu depăși numărul maxim de câmpuri asociate pentru o tabelă, acesta fiind de circa 255. Se mai poate folosi și în cazul în care dorim despărțirea elementelor fixe, a informațiilor care se modifică mai rar față de cele care se modifică destul de des.

Un **exemplu** ar fi la aplicația pe care o proiectăm noi, împărțirea informațiilor despre studenți în două entități: informațiile personale fiind stocate în tabelul StudPersonal și informațiile care se modifică de la an la an în tabela Studenti.

Pentru a realiza efectiv această relație trebuie introdus atributul CodStud și în tabela StudPersonal. Astfel cele două se transformă astfel:

- **Studenti** cu atributele: **CodStud**, An, Grupa, Media, Bursa
 - **StudPersonal** cu atributele: CodStud, **CNP**, Nume, Init, Prenume, Data Nasterii, Loc Nașterii, Tata, Mama, Adresa
- Informațiile stocate așa sunt și mai ușor de manipulat.

2.3.2. Relația „unul-la-multe” (1-N sau one to many)

Această relație este o relație prin care unui element din mulțimea E1 îi corespund unul sau mai multe elemente din mulțimea E2, dar unui element din mulțimea E2 îi corespunde un singur element din mulțimea E1 (Figura 3.2.).

O atenție sporită trebuie să avem la specificarea părților acestui tip de relație.

Un **exemplu** al acestui tip de relație din cadrul aplicației pe care o proiectăm noi este că într-o facultate sunt mai mulți studenți, și se precizează că un student aparține unei singure facultăți din cadrul aceleiași universități.

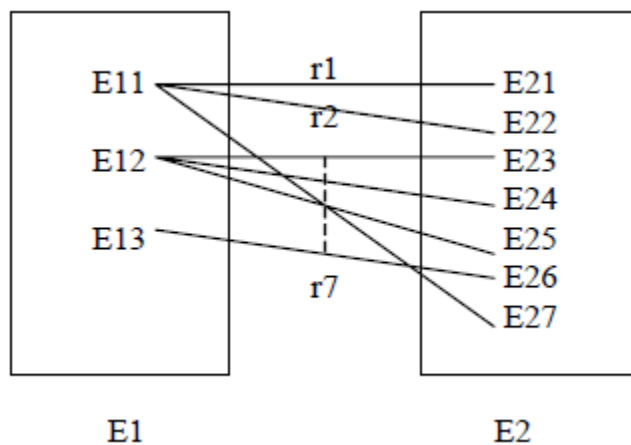


Figura 3.2. Relația „unul-la-multe”

Pentru a realiza efectiv această relație trebuie introdus atributul CodFac și în tabela Studenti. Astfel cele două se transformă astfel:

- **Facultate** cu atributele: **CodFac**, Denumire, Adresa, Nume Decan
- **Studenti** cu atributele: **CodStud**, CodFac, An, Grupa, Media, Bursa

Observație: Dacă se adăuga atributul CodStud în tabela Facultate, se păstrau date redundante în această tabelă, adică pentru fiecare student al facultății trebuiau păstrate informații referitoare la facultate: denumirea ei, adresa, numele decanului, etc.

2.3.3. Relația „multe-la-multe” (M-N sau many to many)

Această relație este o relație prin care unui element din mulțimea E1 îi corespund unul sau mai multe elemente din mulțimea E2, și reciproc (Figura 3.3.).

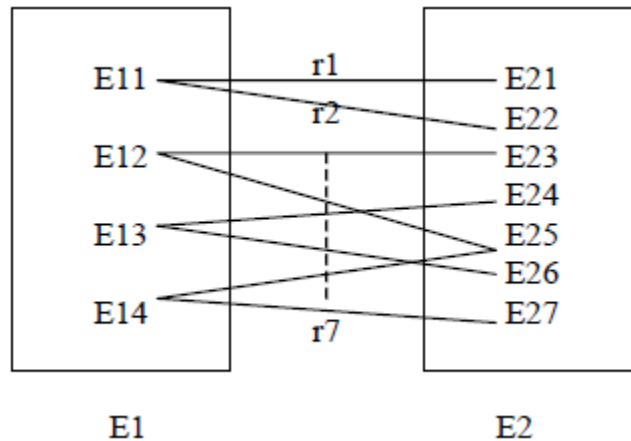


Figura 3.3. Relația „multe-la-multe”

Acest tip de relație este foarte des întâlnită, dar nu poate fi implementată în bazele de date relaționale. De fapt, pentru modelarea acestei relații se folosește o relație suplimentară, de tip unul-la-multe pentru fiecare din relațiile inițiale.

Un **exemplu** al acestui tip de relație din cadrul aplicația pe care o proiectăm noi este că un student participă la mai multe materii, cursuri, iar o materie este frecventată de mai mulți studenți.

Pentru a soluționa această problemă am introdus o tabelă suplimentară numită **Note**, care va face legătura între tabelele Materii și Studenți. Tabelele inițiale se modifică astfel:

- **Studenți** cu attributele: **CodStud**, CodFac, An, Grupa, Media, Bursa
- **Materii** cu attributele: **CodMaterie**, Denumire, An, Nume Profesor
- **Note** cu attributele: CodStud, CodMaterie, Nota, Data

Prin introducerea atributului Data în tabela Note s-a soluționat problema care apare atunci când un student trebuie să fie examinat de mai multe ori până la promovarea materiei respective.

2.3.4. Relația unară

Toate relațiile prezentate anterior sunt relații binare, având câte două relații implicate. Relațiile unare folosesc doar o singură relație, aceasta fiind asociată cu ea însăși (Figura 3.4.).

Exemplul clasic al acestei relații unare este cazul managerului unei companii, care la rândul său este tot un angajat al acelei companii. Relațiile unare se modelează la fel ca și relațiile binare.

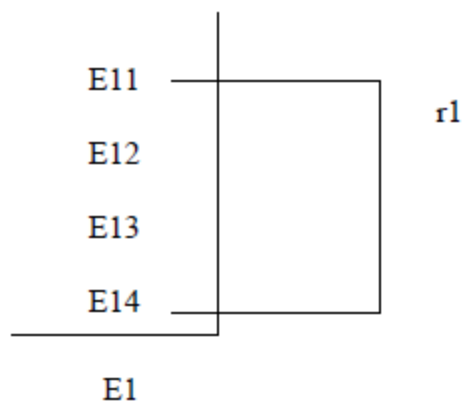
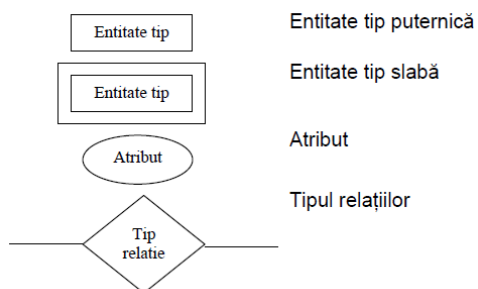


Figura 3.4. Relația unară

2.4. Diagrama entitate-relație

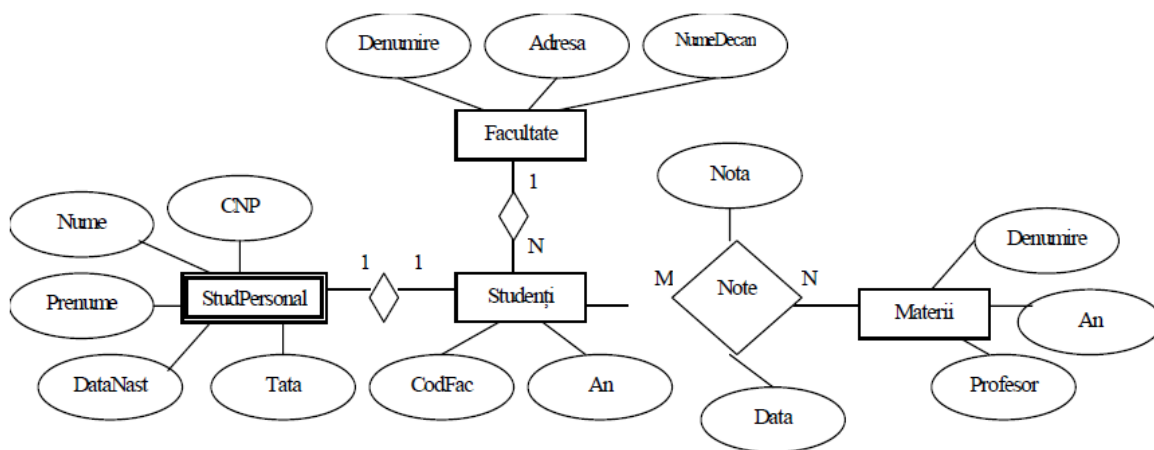
Diagrama entitate-relație este modelul entitate-relație reprezentat. prin mulțimile de entități și relații dintre acestea.

Există mai multe variante de notații pentru redarea acestei diagrame, astfel:



Exemplu:

Se exemplifică dezvoltarea modelului conceptual de nivel înalt al bazei de date Universitate.



2.5. Constrângeri de integritate

Constrângerile de integritate sunt reguli care se definesc la proiectarea unei baze de date și care trebuie să fie respectate de-a lungul existenței acesteia.

Entitățile unei baze de date reflectă realitatea modelată și de aceea valorile pe care le conțin trebuie să respecte anumite reguli, care să corespundă celor din realitate.

Vom folosi în continuare pentru termenul de entitate denumirea tabelă.

Constrângerile se pot clasifica astfel:

- în cadrul tablei;
- sau între table.

Constrângerile din cadrul unei table sunt reguli care se impun în cadrul unei singure table și asigură integritatea datelor acesteia. Ele sunt de 3 categorii:

- **constrângeri de domeniu.** Aceste constrângeri sunt condiții care se impun valorilor atributelor și asigură integritatea domeniilor atributelor.
- **constrângeri de nuplu** (de înregistrare din tabelă). Aceste constrângeri sunt condiții care se impun nuplurilor unei entități (înregistrărilor din tabelă) și asigură identificarea corectă a nuplurilor prin intermediul cheilor primare.
- **constrângeri impuse de dependențe de date** (dependențe funcționale). Acestea sunt constrângeri prin care valorile unor atribute ale unei entități (câmpuri ale tablei) determină valorile altor atribute ale aceleiași entități.

Constrângerile între table sunt reguli care se impun între două sau mai multe relații. Cele mai importante sunt constrângerile de integritate referențială, care se realizează prin intermediul cheilor străine și asigură asocierea corectă a tabelelor.

2.5.1. Constrângerile de domeniu

Constrângerile de domeniu sunt condiții impuse valorilor atributelor pentru ca acestea să corespundă semnificației pe care o au în realitatea modelată.

În reprezentarea unei entități printr-un tabel, valorile atributelor sunt reprezentate pe coloane. Din această cauză aceste constrângeri se mai numesc și constrângeri de coloană.

Sunt 3 tipuri de constrângeri de coloană:

- *Constrângerea NOT NULL.* Valoarea NULL este o valoare particulară, care nu reprezintă valoarea 0, ci lipsă de informație. Această valoare NULL poate apărea când nu se cunosc respectivele informații, ca de exemplu, în aplicația proiectată de noi, în tabela StudPersonal nu se cunoaște numele tatălui (această informație nu este esențială).

Nu orice atribut poate lua valoarea NULL, ca exemplu, numele unui student, pentru ca nu ar avea sens înregistrarea unui student al cărui nume nu se cunoaște. În astfel de situații la definirea relațiilor se impune atributului constrângerea NOT NULL, însemnând că acest atribut nu poate lua valoare NULL în orice înregistrare din tabelă;

- *Constrângerea DEFAULT.* Această constrângere este folosită pentru stabilirea unei valori implicite (DEFAULT) pentru un atribut al entității. În cazul în care la inserarea unui nuplu (înregistrări) nu se specifică valoarea unui atribut (câmp), atunci acesta primește valoarea implicită (dacă a fost definită) sau valoarea NULL (dacă nu a fost definită o valoare implicită pentru atributul respectiv, dar sunt admise valori NULL). Dacă nu a fost definită o valoare implicită și nici nu sunt admise valori NULL se generează o eroare;

- *Constrângerea CHECK.* Constrângerea CHECK este după cum îi spune și numele o constrângere de verificare. În limbajul SQL, domeniile în care pot lua valori atributele se pot stabili ca tipuri de date predefinite. Pentru fiecare atribut se pot adăuga constrângeri de verificare la definirea tabelului.

2.5.2. Constrângerile referitoare la n-upluri (înregistrările din tabelă)-Cheia primară

O entitate este definită ca o mulțime de n-upluri. Deci, n-uplurile entității trebuie să fie distincte, acest lucru însemnând că într-o entitate nu pot exista două sau mai multe n-upluri care să conțină aceeași combinație de valori pentru fiecare atribut.

O cheie primară a unei entități (tabele) este o submulțime de attribute ale entității care are următoarele proprietăți:

- este unică, adică orice combinație de valori ale atributelor acestei chei este unică pentru orice stare a relației;
- este stabilă, adică informația corespunzătoare ei nu se modifică niciodată prin operații de actualizare a datelor;
- nu se admit valori NULL pentru nici unul din attributele cheii respective.

În concluzie, o cheie primară reprezintă unul sau mai multe câmpuri ale tabelului care identifică unic fiecare înregistrare din tabela respectivă.

2.5.3. Constrângeri între tabele

Relațiile dintre tipurile de entități definite în modelul conceptual al unei baze de date se realizează în modelul relațional prin intermediul cheilor străine.

O cheie străină este o submulțime de attribute ale unei entități E1 care referă entitatea E2 și îndeplinește următoarele condiții: attributele cheii străine din E1 sunt definite pe domenii compatibile cu cele ale atributelor cheii din entitatea E2, și cheia din entitatea E2 este cheie primară în această relație.

Această cheie străină determină o asociere între câmpurile unor tabele cu cele ale altei tabele și creează abilitatea de realizare a unirii tabelurilor respective prin intermediul operațiilor JOIN.

Integritatea referențială este proprietatea bazei de date care garantează că oricare valoare a unei chei străine se regăsește printre valorile cheii corespunzătoare din relația referită, sau cheia străină are valoarea NULL (dacă attributele acesteia nu sunt supuse constrângerii NOT NULL).

Exemplu:

Revenind la baza de date proiectată de noi, numită Universitate vom defini cheile primare și cele străine din cadrul fiecărei tabele. Cheile primare vor fi subliniate pentru o mai clară interpretare.

- Pentru tabela **Faculate** vom presupune:

cod facultății (CodFac) este unic pentru fiecare facultate din cadrul universității respective.

Tabela Facultate va avea următoarea *structură*:

Facultate (CodFac, Denumire, Adresa, Nume Decan)

- Pentru tabela **StudPersonal** dispunem de mai multe opțiuni în alegerea cheii primare.

Ea poate fi reprezentată de CNP acesta fiind teoretic unic fiecărei persoane.

Din păcate s-au întâlnit cazuri în care două persoane au același CNP și pentru a evita un posibil conflict, adăugăm la tabela un alt atribut numit CodStud și va fi un număr unic pentru fiecare înregistrare din tabela StudPersonal.

Tabela StudPersonal va avea următoarea structură:

StudPersonal (CodStud, CNP, Nume, Init, Prenume, DataNasterii, LocNașt, Tata, Mama, Adresa)

- Pentru început la tabela **Studenti** inițială ar trebui introdus câmpul CodFac. Acesta va reprezenta o cheie străină a aceste tabele și va face legătura directă între tabela Studenti și tabela Facultate.

Tabela Studenti va avea următoarea structură:

Studenti (CodFac, CodStud, An, Grupa, Media, Bursa).

Vom stabili cheia primară a acestei tabele. În acest caz, cheia primară trebuie să fie o cheie compusă din attributele CodFac și CodStud pentru a identifica unic fiecare înregistrare din tabelă. Dacă am fi ales numai câmpul CodFac nu ar fi fost bine deoarece la o facultate corespund mai mulți studenți și atunci ar apărea CodFac același pentru mai multe înregistrări și nu ar fi îndeplinite condițiile pentru ca un câmp să fie cheie primară.

La fel s-ar fi întâmplat și dacă am fi optat pentru câmpul CodStud deoarece acest cod este un număr unic pentru fiecare student dintr-o facultate, același cod putând fi asociat și pentru un alt student de la o altă facultate.

Tabela Studenti va avea următoarea structură:

Studenti (CodFac, CodStud, An, Grupa, Media, Bursa)

- Pentru tabela Materii vom presupune:

codul materiei (CodMaterie) este unic pentru fiecare disciplină din cadrul universității respective.

Tabela Materii va avea următoarea structură:

Materii (CodMaterie, Denumire, An, Nume Profesor)

- În tabela **Note**, care are inițial două atribute Nota și Data trebuie introduse mai multe câmpuri. Unul ar fi câmpul CodStud, care va reprezenta o cheie străină și va face legătura cu tabela Studenti pentru a cunoaște în orice moment cărui student îi aparține nota respectivă. Un alt câmp care ar trebui introdus este câmpul CodMaterie, care va reprezenta o cheie străină și va face legătura cu tabela Materii pentru a cunoaște în orice moment ce notă i-a fost acordată studentului respectiv.

Tabela Studenti va avea următoarea structură:

Note (CodStud, CodMaterie, Nota, Data).

Vom stabili cheia primară a acestei tabele. În acest caz, cheia primară ar putea fi o cheie compusă din attributele CodStud, CodMaterie și Data pentru a identifica unic fiecare înregistrare

din tabelă. Câmpul Data trebuie introdus în cheia primară deoarece un student poate să participe la un examen de mai multe ori.(fie că a fost absent, fie că nu a obținut notă de trecere, fie că dorește o mărire a notei respective). Cum este greu de manipulat o astfel de cheie primară, se propune introducerea unui alt câmp în tabelă numit CodNota, care va fi unic pentru fiecare înregistrare din tabelă. Astfel, în concluzie, tabela Note va avea următoarea structură:

Note (CodNota, CodStud, CodMaterie, Nota, Data).

2.6. Dependente funcționale

Dependența funcțională definește relația dintre un atribut sau un grup de atribute ale unui tabel și un alt atribut sau grup de atribute ale altuia. După cum am specificat, atributele se refră la câmpurile tabelii. Prin urmare trebuie să vedeți ce câmpuri depind de alte câmpuri.

În orice tabelă pot exista două categorii de dependente funcționale:

- Dependente funcționale determinate de cheile tabelii; astfel de dependente funcționale nu produc redundanța datelor și nici anomalii de actualizare a relației
- Dependente funcționale în care atributul determinat nu este o cheie a tabelii; astfel de dependente funcționale produc redundanța datelor și anomalii de actualizare a tabelii.

Constrângerile de cheie sunt constrângeri implicite, conținute în definiția relației și sunt verificate și impuse automat de sistemul de gestiune; proiectantul bazei de date nu trebuie să prevadă nimic suplimentar pentru ca aceste constrângeri să fie satisfăcute de orice stare a relației. În schimb, dependențele funcționale în care atributul determinant nu este o cheie a relației sunt constrângeri explicite, care nu sunt verificate și nici impuse de sistemul de gestiune.

Verificarea și impunerea acestor dependente funcționale se poate face numai procedural, prin triggere, proceduri stocate sau funcții impuse în programele de aplicație.

2.7. Normalizare. Forme normale

La proiectarea bazelor de date relaționale se stabilesc entitățile din realitatea modelată. Modul în care se pot stabili entitățile unei baze de date nu este unic și de aceea este necesar să existe criterii de evaluare a calității entităților, astfel încât acestea să asigure integritatea datelor.

Procesul de normalizare propus de E.F. Codd în 1970 urmărește execuția asupra unei tabele a unor serii de teste pentru a cerceta apartenența la forma normală.

Codd propune trei forme normale (3NF), cea mai bună definiție fiind dată mai târziu de Boyce și Codd, fiind cunoscută sub numele de forma normală Boyce-Codd.

Normalizarea datelor poate fi privită ca un proces în timpul căruia schemele tabelă nesatisfăcătoare sunt descompuse prin împărțirea atributelor în tabele cu atribute mai puține ce posedă proprietățile dorite.

În fond, unul din obiectivele procesului de normalizare este asigurarea faptului că tabela posedă o bună construcție asigurând posibilități de modificare cu eliminarea anomaliilor care pot apărea.

Forma normală oferă proiectantului bazei de date :

- un schelet formal pentru analiza relațiilor bazat pe chei și pe dependența funcțională între atribute;
- serie de teste ce pot elimina tabelele individuale astfel încât baza de date relațională poate fi normalizată în orice grad.

Când un test nu este trecut, tabela va fi descompusă în tabele ce trec testele de normalitate

2.7.1. Forma normală de ordin 1 (FN1)

Forma normală de ordin 1 este considerată ca fiind parte a definiției formale a unei tabele.

Ea nu permite attribute cu mai multe valori, attribute compuse sau combinații ale lor. Aceasta stabilește ca domeniul atributelor trebuie să includă numai valori atomice și valoarea oricărui atribut într-un nuplu este o valoare unică în domeniul atributului respectiv.

Deci, FN1 nu permite un set de valori, un nuplu de valori sau o combinație a acestora ca valoare a unui atribut pentru un nuplu. Cu alte cuvinte, FN1 nu permite tabele în tabele sau tabele ca attribute ale nuplurilor. Valorile permise de FN1 sunt atomice sau indivizibile, pentru un domeniu specificat de valori.

Exemplu:

Considerăm că în tabela:

Materii (CodMaterie, Denumire, An, NumeProfesor),

unde cheia primară este **CodMaterie** este introdusă o înregistrare de tipul:

CodMaterie	Denumire	An	NumeProfesor
1	Analiză matematică	1	O.Stanășilă, P.Flondor, M.Olteanu

Această înregistrare reprezintă o disciplină care este predată de trei profesori diferiți. Acest nuplu (înregistrare) nu îndeplinește FN1, deoarece la atributul NumeProfesor nu sunt valori atomice, ci un set de valori.

Pentru a rezolva această problemă vom introduce mai multe înregistrări, care vor îndeplini cerințele FN1, considerând trei materii diferite astfel:

CodMaterie	Denumire	An	NumeProfesor
1	Analiză matematică	1	O.Stanășilă
2	Analiză matematică	1	P.Flondor
3	Analiză matematică	1	M.Olteanu

2.7.2. Forma normală de ordin 2 (FN2)

A doua formă normală impune ca fiecare atribut (coloană) să fie dependent de fiecare parte a cheii principale.

Mai exact, o tabelă îndeplinește FN2 dacă îndeplinește FN1 și conține numai attribute care dau informații despre cheia tabeli.

Exemplu:

Considerând că în tabela:

Materii (CodMaterie, Denumire, An, NumeProfesor),

unde cheia primară este **CodMaterie** că ar mai exista și alte câmpuri cum ar fi: Nume Student, Nota, Data Examinării.

Acea structură nu ar fi proiectată bine, neîndeplinind FN2. Pentru a soluționa această problemă trebuie împărțită acea tabelă în mai multe, astfel:

Studenti (CodStud, Nume,...)

Materii (CodMaterie, Denumire, An, NumeProfesor),

Note (Nota, Data, CodStud, Cod Materie)

2.7.3. Forma normală de ordin 3 (FN3)

Pentru a ajunge la a treia formă normală, tabelul trebuie să fie deja în prima și a doua formă normală. Pentru a fi în a treia formă normală, trebuie ca toate câmpurile non-primare să depindă numai de câmpurile primare.

Deși nu face parte în mod riguros din normalizare, de obicei nu este recomandabil să includeți câmpuri care pot fi derivate din alte câmpuri situate în același tabel sau în tabelele aflate în relație.

Exemplu:

Să considerăm baza de date Universitate, mai exact tabela StudPersonal, care are structura:

StudPersonal (CodStud, **CNP**, Nume, Init, Prenume, DataNasterii, LocNașt, Tata, Mama, Adresa)

Nu are rost să stocăm un alt câmp numit Vârsta, care se poate calcula din DataNasterii.

2.7.4. Forma normală Boyce-Codd (FNBC)

Forma normală Boyce-Codd este o formă strictă FN3, înțelegând prin aceasta că fiecare tabelă FNBC este în același timp o tabelă FN3, cu toate că o tabelă FN3 nu este în mod necesar și o tabelă FNBC. Cele două forme sunt asemănătoare, ambele impunând condiția ca atributul care determină funcțional alte atribute să fie o cheie a tabelii. Forma normală Boyce-Codd este mai restrictivă decât FN3, deoarece în FNBC se impune această condiție tuturor atributelor, prime sau neprime, pe când în FN3 condiția se impune numai atributelor neprime. Atributele prime sunt atributele care aparțin unei chei, iar celelalte se numesc atribute neprime.

Orice tabelă formată din două atribute este FNBC, FN2 și FN3.

Această tabelă compusă din două atribute este FN2, deoarece, fie cheia este formată din ambele atribute și atunci nu există atribute neprime, fie cheia este formată dintr-unul din atribute, iar dependența funcțională a celui alt atribut (care este atribut neprim) față de cheie este totală.

Această tabelă compusă din două atribute este FN3 deoarece este FN2 și nu poate exista nici un atribut neprim care să determine funcțional un alt atribut neprim, deoarece o tabelă cu două atribute nu poate avea decât cel mult un atribut neprim.

S-ar putea să vă simțiți copleșiți de aceste reguli. De fapt, pe măsură ce veți dobândi experiență, veți începe să creați fișiere normalizate de la bun început.

2.8. Structuri de indecși în tabelele de date

Un index reprezintă o cale rapidă de localizare a înregistrărilor dintr-o tabelă, prin gruparea tuturor înregistrărilor pentru un anumit atribut sau grup de attribute.

Indexarea este utilizată în două scopuri principale:

- accelerarea căutărilor în baza de date;
- asigurarea unicității înregistrărilor.

Vom privi o relație ca o colecție de date (o mulțime) în care nu sunt admise elemente duplicat. În cazul unei mulțimi reprezentate printr-o colecție neordonată de elemente, timpul de căutare a unui element crește proporțional cu numărul de elemente ale mulțimii, deoarece în cazul cel mai rău trebuie parcurse toate elementele mulțimii pentru a găsi elementul dorit.

Timpul de căutare a unui element poate fi micșorat considerabil dacă elementele mulțimii sunt ordonate.

Un exemplu este cel utilizat uzual în cărți. Într-o carte găsim la sfârșit termenii importanți aranjați în ordine alfabetică. La fiecare termen din această listă este furnizată un număr de pagină în care apare și este explicat termenul. Utilizând această listă se găsește imediat un termen căutat. Fără o astfel de listă, neexistând o ordine de ghidare a căutării, singura alternativă este explorarea completă a întregului material pentru a găsi termenul dorit.

În general, operațiile de căutare, inserare și ștergere a elementelor într-o mulțime (tabelă) se execută mai rapid dacă elementele mulțimii (înregistrările) sunt reprezentate printr-o colecție ordonată. În tehnologia bazelor de date, ordonarea colecțiilor de date se face prin indexarea datelor.

Indexul unei tabele este o structură de date adițională memorată în baza de date care permite accesul rapid la înregistrările tabelii prin ordonarea acestora.

De fapt, indexul poate fi gândit ca o tabelă cu două attribute: primul atribut conține valorile atributelor tabelii bazei de date pentru care se crează indexul, iar al doilea conține un pointer la locația nuplurilor corespunzătoare. Valorile sunt aranjate fie în ordine descendentă cheii de indexare, fie în ordine ascendentă.

O reprezentare generică a structurii de index este data în Figura 3.5.

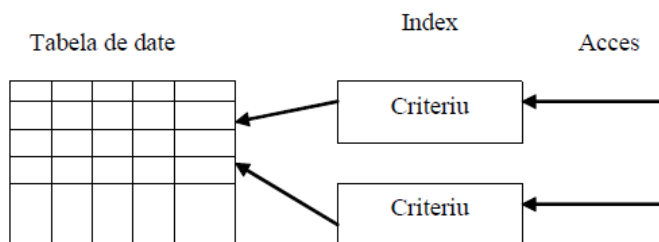


Figura 3.5. O structură generică de index

Indecșii se clasifică după tipul de câmp sau după nivel și după modul de organizare a tabelii. O clasificare a acestora este următoarea:

1. **Indexul primar** este un index asociat unei tabele ordonate după câmpul cheie al tabelii, iar în structura de index se utilizează câmpul cheie.
2. **Indexul secundar** este un index construit tot pe baza unui câmp cheie, dar tabela nu este ordonată după câmpul cheie.

3. **Indexul de grup** (cluster) este un index construit după câmpuri ce nu sunt câmpuri cheie (criteriu de acces este diferit de câmpul cheie), iar tabela poate fi ordonată sau nu relativ la criteriul de acces.

4. **Indexul multinivel** (se mai numesc și indecși de blocuri) se aplică oricăror tabele. Principiul de bază este de a construi niveluri de indexare până când structura adițională de date corespunde indexului de cel mai mare nivel poate fi memorată într-un singur bloc. Acești indecși se construiesc în două forme: bazați pe arbori B și bazați pe arbori B+.

2.8.1. Indexul primar

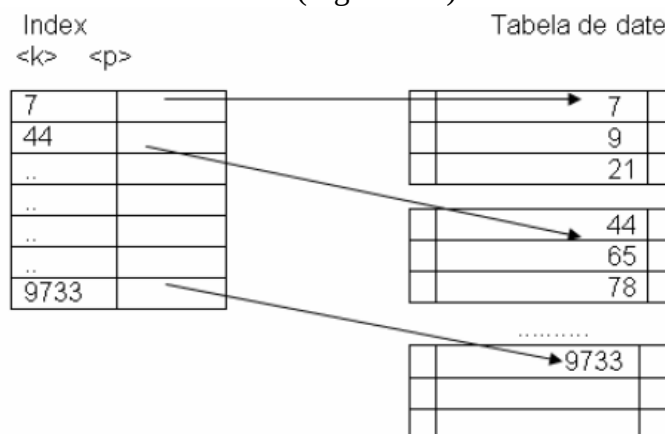
Un index primar este un fișier ordonat cu înregistrări de lungime fixă având două câmpuri. Primul câmp al indexului este de același tip cu un câmp cheie ordonat al tabelului de date, iar al doilea câmp este un pointer către un bloc (o adresă a unui bloc).

Câmpul cheie de ordonare se mai numește și *cheie primară* a tabelului de date. Asociația celor două câmpuri formează intrarea index sau înregistrarea index pentru fiecare bloc al tabelului de date.

Cum tabela de date este ordonată după valorile câmpului index, în fișierul index valoarea primului câmp este dată de valoarea câmpului index de la prima înregistrare a blocului. Al doilea câmp, cel ce semnifică un pointer este de tip întreg și indică adresa blocului.

Volumul datelor în index este mai mic datorită faptului că în index avem o singură intrare pentru un bloc, cât și datorită faptului că un index este similar cu o tabelă, dar are numai două câmpuri.

Ca efect, căutarea într-un fișier index este mult mai rapidă decât într-o tabelă de date, putând fi utilizate metode de căutare binare. (Figura 3.6.)



<k> reprezintă cheia de ancorare bloc

<p> reprezintă pointerul la bloc

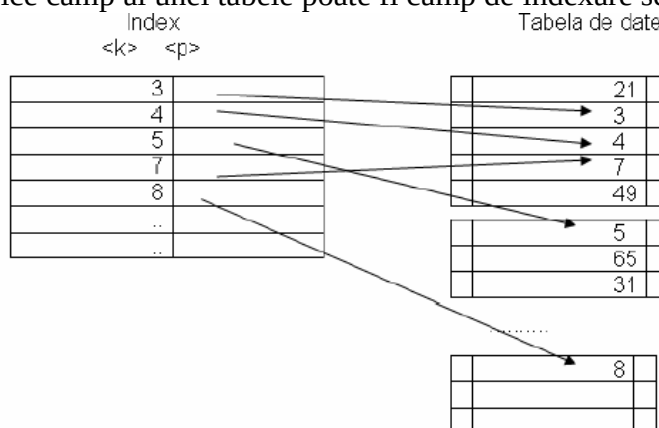
Figura 3.6. Indexul primar

2.8.2. Indexul secundar

Metoda de indexare secundară se aplică la tabele neordonate, indiferent dacă valorile câmpului după care se face indexarea în tabela de date sunt sau nu distincte.

Indexul secundar este un fișier ordonat cu două câmpuri ca și la alți indecși, în care primul câmp este identic cu cel al tabelului de date, iar al doilea câmp este un pointer. Câmpul pentru care indexul este construit se numește și câmp de indexare.

În concluzie, orice câmp al unei tabeli poate fi câmp de indexare secundar. (Figura 3.7.)



<k> reprezintă cheia de ancorare bloc

<p> reprezintă pointerul la bloc

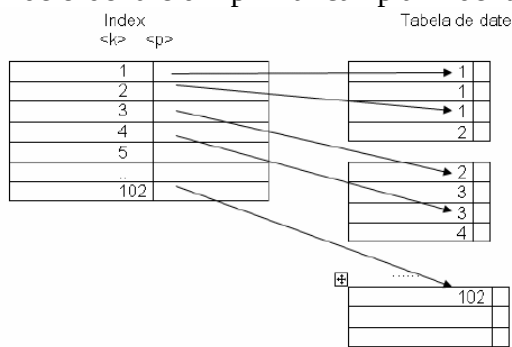
Figura 3.7. Indexul secundar

2.8.3. Indexul de grup

Astfel de indecși sunt folosiți când înregistrările tabelului de date sunt ordonate fizic după un câmp care nu este cheie (noncheie), deci un câmp ce nu are valori distincte la fiecare înregistrare. Un astfel de câmp identifică un grup de înregistrări (clustering field). În această situație se poate crea un index ce facilitează găsirea înregistrărilor ce aparțin unui câmp.

Un index de grup este deci un fișier ordonat cu două câmpuri, primul câmp conținând aceeași informație cu cea a câmpului noncheie de ordonare, al doilea fiind destinat unui pointer către un bloc de date. În acest mod, fișierul index conține câte o intrare pentru fiecare valoare distinctă a câmpului de ordonare.

Al doilea câmp al înregistrării index conține un pointer către blocul în care apare pentru prima oară valoarea câmpului de ordonare din primul câmp al indexului. (Figura 3.8.)



<k> reprezintă cheia de ancorare bloc

<p> reprezintă pointerul la bloc

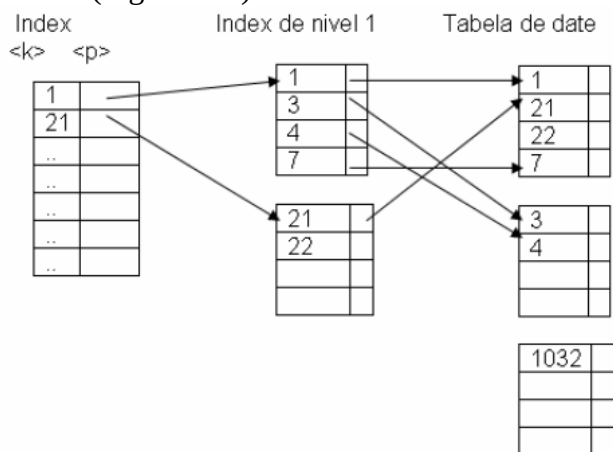
Figura 3.8. Indexul de grup

2.8.4. Indexul multinivel

Metodele de indexare descrise până acum operează cu un fișier index ordonat. Asupra fișierului index se aplică metode de căutare binară pentru localizarea înregistrărilor cu valoarea specificată în câmpul index.

Pentru un index multinivel, fișierul index este văzut ca un nou fișier la care se construiește un nou index și așa mai departe.

Primul fișier index conține câte o valoare distinctă pentru fiecare cheie de indexare. Se poate crea un index primar pentru primul nivel, nivel numit și nivel secund al indexului multinivel. Cum al doilea nivel este un index primar se poate folosi metoda de ancorare a blocurilor, așa că al doilea nivel are câte o intrare pentru fiecare bloc al primului nivel, întrucât este în esență un index primar. (Figura 3.9.)



<k> reprezintă cheia de ancorare bloc

<p> reprezintă pointerul la bloc

Figura 3.9. Indexul cu două niveluri

În imaginea din figura 3.10., este prezentată baza de date, după ce s-a realizat proiectarea ei, crearea tabelurilor și a relațiilor dintre ele și după ce s-au impus constrângerile.

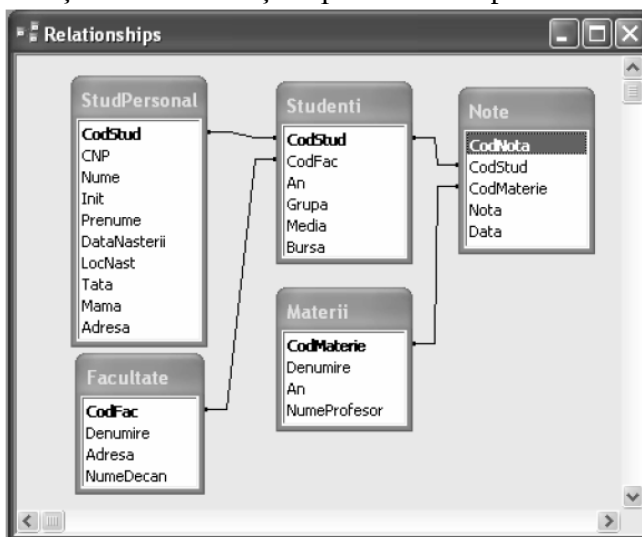


Figura 3.10. Baza de date

Câmpurile scrise îngroșat sunt cheile primare din cadrul fiecărei tabele.

Liniile dintre tabele arată relațiile care le-am creat prin proiectare între fiecare două tabele. La capetele liniilor sunt două câmpuri: câmpul scris neîngroșat reprezintă cheia străină a tabeli și după cum spune și definiția, face legătura cu cheia primară a celeilalte tabele.

3. Desfășurarea lucrării

Se vor testa exemplele prezentate.

Se va realiza un exemplu liber ales.